

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH

BÁO CÁO BÀI TẬP LỚN
MÔN: THIẾT KẾ CƠ SỞ DỮ LIỆU

Đề tài: Thiết kế Cơ sở Dữ liệu cho Nền tảng Dịch vụ Tín dụng & Điểm thưởng cấp doanh nghiệp (OctaPoint CaaS)

Nhóm sinh viên thực hiện:

STT	Họ và tên	Lớp
1	Lưu Minh Hoà	Lớp: 012012100203
2	[Họ và tên 2]	Lớp: 012012100203
3	Trần Thị Ngọc Duyên	Lớp: 012012100203
4	[Họ và tên 4]	Lớp: 012012100203
5	[Họ và tên 5]	Lớp: 012012100203

TP. Hồ Chí Minh, Năm 2026

Mục lục

1	Khảo sát Yêu cầu và Nghiệp vụ Hệ thống	3
1.1	Tổng quan về bài toán và phạm vi đề tài	3
1.1.1	Bối cảnh thực tế và lý do phát triển	3
1.1.2	Mục tiêu của hệ thống OctaPoint	3
1.1.3	Phạm vi dữ liệu của đề tài	4
1.1.4	Đối tượng sử dụng và Yêu cầu chức năng	4
1.1.5	Yêu cầu phi chức năng và Tác động đến thiết kế CSDL	6
1.1.6	Giới hạn phạm vi đề án	7
1.1.7	Quy tắc nghiệp vụ trọng yếu và Ảnh xạ dữ liệu	8
2	Thiết kế Mô hình Dữ liệu Quan niệm	9
2.1	Xác định Các Thực thể và Thuộc tính	9
2.1.1	Phân hệ Tổ chức và Phân quyền (Tenant & RBAC)	9
2.1.2	Phân hệ Tín dụng và Điểm thưởng (Loyalty Engine)	10
2.1.3	Phân hệ Sổ cái Giao dịch và Kiểm toán (Ledger & Audit)	11
2.2	Xác định các mối quan hệ (Relationships)	11
2.3	Sơ đồ Thực thể Kết hợp (ERD)	12
3	Thiết kế Cơ sở dữ liệu Logic và Chuẩn hóa	13
3.1	Chuyển đổi từ Mô hình Quan niệm sang Lược đồ Logic	13
3.1.1	Các quy tắc chuyển đổi áp dụng	13
3.1.2	Danh sách Lược đồ Quan hệ sơ bộ	14
3.2	Phụ thuộc hàm và Khóa ứng viên	14
3.2.1	Tính Bao đóng và Tìm khóa ứng viên	15
3.3	Chuẩn hóa Dữ liệu (Normalization)	15
3.3.1	Kiểm chứng Dạng chuẩn Boyce-Codd (BCNF)	15
3.4	Sơ đồ Lược đồ Logic (Logical Schema Diagram)	16
3.5	Từ điển Dữ liệu (Data Dictionary)	17
3.5.1	Bảng TENANT (Đơn vị thuê bao)	17
3.5.2	Bảng MERCHANT (Doanh nghiệp đối tác)	17
3.5.3	Bảng USER (Tài khoản nhân sự)	17
3.5.4	Bảng ROLE (Vai trò)	18
3.5.5	Bảng CUSTOMER (Khách hàng)	18
3.5.6	Bảng CREDIT (Số dư ví điểm)	18
3.5.7	Bảng CAMPAIGN (Chiến dịch)	19
3.5.8	Bảng CAMPAIGN_MERCHANT (Phạm vi áp dụng)	19
3.5.9	Bảng REWARD_RULE (Quy tắc tính điểm)	19
3.5.10	Bảng CREDIT_TRANSACTION (Sổ cái giao dịch điểm)	20

3.5.11	Bảng API_CREDENTIAL (Khóa tích hợp)	20
4	Thiết kế Vật lý và Triển khai	21
4.1	Đặc tả Workload và Chọn tổ chức lưu trữ	21
4.1.1	Đặc tả tải công việc (Workload)	21
4.1.2	Lựa chọn tổ chức lưu trữ (Storage Organization)	21
4.2	Ánh xạ Kiểu dữ liệu và Ràng buộc (Đầu vào)	22
4.2.1	Ánh xạ Kiểu dữ liệu (Data Type Mapping)	22
4.2.2	Ràng buộc Toàn vẹn (Constraints)	22
4.3	Chiến lược Đánh chỉ mục (Indexing)	22
4.3.1	Cơ sở lý thuyết và Thuật toán lựa chọn	23
4.3.2	Thiết kế Chỉ mục cho OctaPoint CaaS	23
4.4	Mã lệnh tạo bảng (DDL - Data Definition Language)	23
4.5	Tối ưu hóa truy xuất dữ liệu	26
4.5.1	Bài toán truy vấn	26
4.5.2	Cây truy vấn trước và sau khi tối ưu	26
4.5.3	Đánh giá thuật toán kết nối (Join Algorithm)	26
4.6	Một số câu truy vấn mẫu	27
4.6.1	Truy vấn 1: Xem lịch sử giao dịch điểm của một khách hàng tại một cửa hàng	27
4.6.2	Truy vấn 2: Tra cứu số dư điểm hiện tại của khách hàng trên toàn bộ các chuỗi (Tenant)	27
4.6.3	Truy vấn 3: Thống kê tổng số điểm đã phát hành của một chiến dịch tại một cửa hàng trong tháng	27
4.6.4	Truy vấn 4: Phân tích khách hàng (Top 5 người có số dư cao nhất tại một Chuỗi)	28
4.6.5	Truy vấn 5: Thống kê hiệu suất các cửa hàng (Tổng điểm đã tiêu/tích lũy theo từng chi nhánh của chuỗi)	28
4.6.6	Truy vấn 6: Tra cứu danh sách các cửa hàng được phép áp dụng một Chiến dịch đang hoạt động	28

Chương 1

Khảo sát Yêu cầu và Nghiệp vụ Hệ thống

1.1 Tổng quan về bài toán và phạm vi đề tài

1.1.1 Bối cảnh thực tế và lý do phát triển

Trong nền kinh tế số hiện nay, các hệ thống tín dụng tiêu dùng và điểm thưởng (loyalty reward systems) đóng vai trò nòng cốt trong chiến lược giữ chân khách hàng tại các chuỗi siêu thị, trung tâm thương mại và dịch vụ F&B. Tuy nhiên, các hệ thống truyền thống vận hành dựa trên cơ sở dữ liệu tập trung đang bộc lộ nhiều hạn chế nghiêm trọng về mặt kiến trúc và vận hành:

- **Nguy cơ bảo mật và toàn vẹn dữ liệu:** Các cơ sở dữ liệu điểm thưởng tập trung truyền thống thường xuyên là mục tiêu của các cuộc tấn công quét mã (API spam), dò mật khẩu (brute-force), đúc điểm trái phép và gian lận từ nội bộ nhân viên.
- **Chi phí và rào cản tích hợp:** Các doanh nghiệp (Merchant) gặp khó khăn trong việc tự xây dựng các hệ thống tín dụng tùy biến, an toàn và minh bạch từ con số không, dẫn đến thời gian triển khai chậm và chi phí bảo trì khổng lồ.
- **Thiếu tính minh bạch và tranh chấp đối soát:** Việc thiếu một hợp đồng thông minh (smart contract) bất biến dẫn đến các tranh chấp về đối soát dữ liệu giữa người tiêu dùng, doanh nghiệp và các đối tác liên minh điểm thưởng chéo.
- **Rào cản tiếp cận Web3:** Các giải pháp điểm thưởng ứng dụng blockchain hiện tại thường ép buộc người dùng và doanh nghiệp phải tự quản lý ví tiền mã hóa, cụm từ khôi phục (seed phrase) và trả phí giao dịch (gas fees), tạo ra rào cản ứng dụng (adoption barrier) rất lớn.

1.1.2 Mục tiêu của hệ thống OctaPoint

Để giải quyết những thách thức trên, đề tài hướng tới việc nghiên cứu và thiết kế cơ sở dữ liệu cho nền tảng **OctaPoint** – Nền tảng Dịch vụ Tín dụng & Điểm thưởng cấp doanh nghiệp (Enterprise-grade Credit-as-a-Service - CaaS).

Hệ thống được thiết kế để tận dụng thông lượng cao của hợp đồng thông minh trên chuỗi khối Sui Layer-1, đồng thời trừu tượng hóa toàn bộ sự phức tạp của blockchain. Mục tiêu cốt lõi của việc thiết kế cơ sở dữ liệu cho OctaPoint bao gồm:

- **Trải nghiệm người dùng Web2 liền mạch:** Ứng dụng kiến trúc xác thực không ví (Walletless Authentication) thông qua Enoki's zkLogin, cho phép người dùng đăng nhập bằng các định danh Web2 quen thuộc (như Google OAuth) mà không cần quản lý khóa riêng tư (private keys).
- **Hỗ trợ Tác tử AI (AI-Ready):** Tích hợp sẵn giao thức Model Context Protocol (MCP) Server, cho phép các trợ lý AI và Bot chăm sóc khách hàng truy vấn số dư tín dụng, kiểm tra lịch sử giao dịch và kích hoạt các chiến dịch tự động một cách an toàn.
- **Kiến trúc đa khách thuê (Multi-tenancy):** Thiết kế lược đồ dữ liệu linh hoạt, phân tách dữ liệu an toàn, cho phép nhiều doanh nghiệp (Merchants) cùng đăng ký, cấu hình chiến dịch tích điểm và quản lý hạn mức độc lập trên cùng một nền tảng.
- **Hiệu năng cao và Khả năng ngoại tuyến (Offline-First):** Tối ưu hóa cơ sở dữ liệu quan hệ (SQL Server) kết hợp bộ đệm (Redis) để duy trì thời gian phản hồi API dưới 100ms, hỗ trợ kiểm tra số dư ngoại tuyến trong các môi trường tần suất cao và đồng bộ ghi chép hàng loạt (batch write-backs) sử dụng Programmable Transaction Blocks (PTBs).

1.1.3 Phạm vi dữ liệu của đề tài

Dựa trên kiến trúc của hệ thống OctaPoint, cơ sở dữ liệu được thiết kế sẽ tập trung quản lý các miền dữ liệu chính phục vụ cho các phân hệ chức năng:

1. **Merchant Web Portal:** Quản lý thông tin doanh nghiệp, thông tin xác thực API, cấu hình chiến dịch điểm thưởng (tỷ lệ tích lũy/đổi thưởng, hệ số nhân, ngày hết hạn) và kiểm soát truy cập (RBAC).
2. **Admin Web System:** Quản lý cấu hình toàn hệ thống, theo dõi kho bạc tài trợ phí gas (Enoki gas sponsorship treasuries), quản lý thanh toán gói cước doanh nghiệp (Subscription/Stripe) và giám sát hạ tầng.
3. **Lớp SDK & MCP (Developer SDKs & MCP Layer):** Quản lý lịch sử giao dịch điểm thưởng bất biến, cung cấp dữ liệu cấu trúc an toàn cho AI Agents thông qua OctaPoint MCP Server.

1.1.4 Đối tượng sử dụng và Yêu cầu chức năng

Hệ thống OctaPoint CaaS phục vụ bốn nhóm tác nhân chính: Quản trị viên hệ thống (System Administrator), Doanh nghiệp đối tác (Merchant Operator), Người dùng cuối (End-User) và Tác tử AI / Hệ thống tự động (AI Agent & SDK System). Việc tách biệt rõ ràng các tác nhân giúp xác định phạm vi truy cập và thiết lập mô hình phân quyền (RBAC) chặt chẽ trong CSDL.

Hình 1.1: Sơ đồ Use Case tổng quan hệ thống OctaPoint

Quản trị viên Hệ thống (System Administrator)

FA-1. **Quản lý tài khoản và phân quyền đa khách thuê (Multi-tenant):** Tạo mới, phê duyệt, cập nhật trạng thái hoạt động của các Công ty mẹ/Chuỗi bán lẻ (Tenants) và các Cửa hàng/Chi nhánh (Merchants) trực thuộc trên nền tảng.

FA-2. **Giám sát hạ tầng Blockchain và Tài nguyên:** Theo dõi ví tài trợ phí gas (Enoki gas sponsorship treasuries), giám sát thông lượng API Gateway và trạng thái hợp đồng thông minh trên mạng lưới Sui.

FA-3. **Quản lý gói cước (Subscription & Billing):** Cấu hình các gói dịch vụ (Tiers), tích hợp với Stripe để quản lý thanh toán tự động và chu kỳ gia hạn của doanh nghiệp.

FA-4. **Truy vấn nhật ký kiểm toán toàn cục:** Theo dõi và truy vết (audit trail) mọi thay đổi cấu hình quan trọng trên nền tảng nhằm đảm bảo tính minh bạch.

Doanh nghiệp Đối tác (Merchant Operator)

FM-1. **Quản lý API Key và Webhook:** Khởi tạo, luân chuyển (rotate) và vô hiệu hóa cặp khóa API/Secret Key phục vụ tích hợp SDK; cấu hình điểm cuối (endpoints) nhận Webhook tại từng chi nhánh.

FM-2. **Cấu hình Chiến dịch & Quy tắc (Rule Engine):** Thiết lập tỷ lệ tích lũy/quy đổi điểm, hệ số nhân theo sự kiện (multiplier campaigns). Quản trị viên cấp Tenant có thể áp dụng chiến dịch cho toàn chuỗi hoặc chỉ định cho một số Merchant nhất định.

FM-3. **Giám sát giao dịch và Đối soát:** Theo dõi luồng điểm phát hành, điểm tiêu lũy, lịch sử giao dịch khách hàng tại cửa hàng và xuất báo cáo đối soát tài chính định kỳ.

Người dùng cuối (End-User)

FU-1. **Xác thực không ví (Walletless Authentication):** Đăng nhập vào hệ thống thông qua các định danh Web2 (Google, Apple OAuth) được hỗ trợ bởi giao thức zkLogin mà không cần quản lý private key.

FU-2. **Truy vấn số dư và Lịch sử:** Xem số dư điểm khả dụng chung theo từng Chuỗi (Tenant) và kiểm tra chi tiết lịch sử giao dịch tại các Cửa hàng (Merchant) thuộc chuỗi đó.

FU-3. **Thực thi giao dịch điểm thưởng:** Thực hiện các giao dịch nhận điểm hoặc thanh toán/đổi quà bằng điểm tại điểm bán (POS) hoặc ứng dụng thương mại điện tử.

Tác tử AI và Hệ thống tự động (AI Agent & SDK System)

FS-1. **Truy vấn ngữ cảnh tự động (MCP Context Retrieval):** Thông qua giao thức MCP, tự động truy xuất dữ liệu số dư, lịch sử giao dịch và quy định chiến dịch để phục vụ ngữ cảnh cho các LLM/Chatbot chăm sóc khách hàng.

FS-2. **Ghi nhận giao dịch thời gian thực:** Gửi các yêu cầu cộng/trừ điểm thưởng thông qua SDK (gRPC/REST) với độ trễ thấp (Sub-100ms).

FS-3. **Đồng bộ trạng thái On-chain/Off-chain:** Lắng nghe và đồng bộ trạng thái giao dịch từ Sui blockchain về cơ sở dữ liệu quan hệ cục bộ (SQL Server).

Bảng 1.1: Tổng hợp yêu cầu chức năng theo vai trò

Mã	Vai trò	Yêu cầu chức năng
FA-1	Administrator	Quản lý tài khoản đa khách thuê (Chuỗi và Cửa hàng)
FA-2	Administrator	Giám sát hạ tầng Blockchain, phí Gas và tài nguyên
FA-3	Administrator	Quản lý gói cước (Subscription), thanh toán định kỳ
FA-4	Administrator	Truy vấn nhật ký kiểm toán (Audit Trail) toàn cục
FM-1	Merchant	Quản lý API Key, Secret và cấu hình Web-hook
FM-2	Tenant/Merchant	Thiết lập chiến dịch, tỷ lệ quy đổi và phạm vi áp dụng
FM-3	Merchant	Giám sát lịch sử giao dịch tại cửa hàng, đối soát
FU-1	End-User	Đăng nhập Web2 qua zkLogin (Không dùng ví Crypto)
FU-2	End-User	Tra cứu số dư ví chung của chuỗi và lịch sử giao dịch
FU-3	End-User	Tích điểm và tiêu điểm tại điểm bán

1.1.5 Yêu cầu phi chức năng và Tác động đến thiết kế CSDL

Các yêu cầu phi chức năng (Non-Functional Requirements - NFR) quy định ràng buộc về chất lượng vận hành của hệ thống CaaS. Đối với khía cạnh Thiết kế Cơ sở dữ liệu, các yêu cầu này định hướng trực tiếp đến việc lựa chọn kiểu dữ liệu, thiết kế chỉ mục (indexing) và chiến lược quản lý giao dịch.

NFR-DB-01 Toàn vẹn dữ liệu & Nhất quán (Integrity & Consistency): Dữ liệu giữa các bảng phải được duy trì nhất quán sau mỗi giao dịch.

- *Hàm ý CSDL:* Bảo đảm thiết lập đầy đủ Khóa chính (PK), Khóa ngoại (FK) với chính sách tham chiếu (CASCADE, RESTRICT) và các ràng buộc dữ liệu (CHECK, UNIQUE). Sử dụng khối giao dịch (Transaction) để đồng bộ với Programmable Transaction Blocks (PTBs) trên Sui.

NFR-DB-02 Bảo mật & Khả năng truy vết (Security & Auditability): Chỉ người dùng có quyền mới được truy cập dữ liệu. Các thay đổi quan trọng phải có khả năng truy vết.

- *Hàm ý CSDL:* Lưu mã băm API Credential (HMAC-SHA256). Các bảng CreditTransaction và AuditLog phải được thiết kế dưới dạng sổ cái bất biến (Immutable Ledger), chỉ cho phép INSERT, tuyệt đối cấm UPDATE/DELETE.

NFR-DB-03 Hiệu năng & Khả năng mở rộng (Performance & Scalability):
Hỗ trợ truy vấn nhanh đối với số dư, giao dịch; đáp ứng sự gia tăng về merchant, customer.

- *Hàm ý CSDL:* Tối ưu hóa chỉ mục (B+ Tree Index) trên các bảng cốt lõi để đảm bảo độ trễ truy vấn dưới 100ms (Sub-100ms). Lược đồ phải hỗ trợ kiến trúc Multi-tenant phân tách dữ liệu an toàn.

NFR-DB-04 Tính sẵn sàng & Khả năng bảo trì (Availability & Maintainability): Dữ liệu duy trì ổn định, cấu trúc rõ ràng, chuẩn hóa và dễ mở rộng.

- *Hàm ý CSDL:* Hệ thống CSDL phải đạt tối thiểu Dạng chuẩn 3 (3NF) hoặc Boyce-Codd (BCNF) nhằm triệt tiêu hoàn toàn dị thường dữ liệu (anomalies) khi thêm, sửa, xóa.

Bảng 1.2: Tổng hợp yêu cầu phi chức năng ảnh hưởng đến CSDL

Mã	Thuộc tính	Yêu cầu CSDL & Cấu hình vật lý
NFR-DB-03	Hiệu năng	Tối ưu B+ Tree Index cho độ trễ truy vấn điểm < 100ms
NFR-DB-02	Bảo mật	Lưu mã băm API Secret, thiết kế phân quyền RBAC đa khách thuê
NFR-DB-01	Tính toàn vẹn	Bảng <code>CreditTransaction</code> bất biến, lưu <code>tx_digest</code> từ blockchain
NFR-DB-04	Khả năng mở rộng	Lược đồ chuẩn 3NF/BCNF hỗ trợ Multi-tenant không gây thắt nút cổ chai

1.1.6 Giới hạn phạm vi đề án

Mặc dù hệ thống thực tế yêu cầu quản lý đa dạng các luồng thông tin, tuy nhiên trong khuôn khổ của môn học Thiết kế Cơ sở dữ liệu, mô hình thiết kế sẽ tập trung sâu vào các thực thể cốt lõi phục vụ nghiệp vụ chính bao gồm: **Tổ chức/RBAC** (Tenant, Merchant, User, Role, API_Credential) và **Khách hàng thân thiết** (Customer, Credit, Campaign, Reward_Rule, Credit_Transaction).

Các nhóm dữ liệu mở rộng như:

- Phân quyền chi tiết (Permission)
- Nhật ký hệ thống và Cảnh báo gian lận (AuditLog, FraudEvent)
- Quản lý gói cước (PricingPlan, Subscription, Invoice)

thuộc phạm vi mở rộng của hệ thống. Các thực thể này được xác định ở mức yêu cầu tổng quan nhưng sẽ được giản lược khỏi Mô hình quan niệm (ERD) và Mô hình logic nhằm tập trung làm rõ luồng giao dịch tích/tiêu điểm chính của hệ thống CaaS.

1.1.7 Quy tắc nghiệp vụ trọng yếu và Ảnh xạ dữ liệu

Các quy tắc nghiệp vụ (Business Rules)

BR-1 Ràng buộc sở hữu Chiến dịch: Một Chuỗi bán lẻ (Tenant) có thể quản lý nhiều Chiến dịch (Campaign). Mỗi Chiến dịch bắt buộc thuộc về duy nhất một Tenant và có thể được áp dụng linh hoạt cho một, nhiều hoặc toàn bộ các Cửa hàng (Merchant) trong chuỗi thông qua bảng trung gian.

BR-2 Ràng buộc độc lập Số dư: Số dư (Credit Balance) của một Khách hàng (Customer) được quản lý tập trung theo từng Chuỗi (Tenant). Nghĩa là một Customer chỉ có một ví điểm duy nhất tại mỗi Tenant, dùng chung cho mọi cửa hàng (Merchant) trực thuộc.

BR-3 Quy tắc Bất biến Giao dịch (Ledger Immutability): Bất kỳ sự thay đổi số dư nào đều sinh ra một bản ghi giao dịch (Credit Transaction) mới, có ghi nhận rõ MerchantID nơi phát sinh. Hệ thống không cho phép sửa đổi hoặc xóa giao dịch đã hoàn tất.

BR-4 Ràng buộc Miền giá trị (Domain Constraints): Tỷ lệ quy đổi, hạn mức và số lượng credit bắt buộc phải là số thực dương (>0). Trạng thái chiến dịch phải thuộc tập giá trị định trước (DRAFT, ACTIVE, EXPIRED).

BR-5 Ràng buộc Mã băm Blockchain: Giao dịch đã đồng bộ On-chain phải lưu trữ mã băm đối soát (sui_tx_digest) dưới dạng duy nhất (UNIQUE) trên cơ sở dữ liệu để phục vụ kiểm toán.

Ảnh xạ Yêu cầu vào Đối tượng Dữ liệu chính

Bảng 1.3: Ảnh xạ Yêu cầu và Nhóm dữ liệu vào Thực thể

Nhóm Dữ liệu	Mã Yêu cầu	Thực thể quản lý & Vai trò
Tổ chức & Dân cư	FR-DB-01, 03	Tenant, Merchant, Customer: Quản lý định danh và hồ sơ đối tác, khách hàng.
Phân quyền	FR-DB-02	User, Role, Permission: Kiểm soát truy cập RBAC.
Loyalty	FR-DB-04, 06	Credit, Campaign, RewardRule, Campaign_Merchant: Quy tắc tích điểm, đổi thưởng và phạm vi áp dụng.
Giao dịch On/Off	FR-DB-05, 07	CreditTransaction: Ghi nhận biến động số dư, tham chiếu cửa hàng và mã băm Blockchain.
Bảo mật & Audit	FR-DB-08, 09	APICredential, AuditLog, FraudEvent: Quản lý API Key, nhật ký hệ thống và chống gian lận.
Thương mại	FR-DB-10	PricingPlan, Subscription, Invoice: Quản lý gói cước và thanh toán của Merchant.

Chương 2

Thiết kế Mô hình Dữ liệu Quan niệm

2.1 Xác định Các Thực thể và Thuộc tính

Dựa trên các phân hệ chức năng và quy tắc nghiệp vụ đã khảo sát, mô hình dữ liệu quan niệm của hệ thống OctaPoint CaaS xác định 11 thực thể cốt lõi cùng danh sách thuộc tính chi tiết.

Để đảm bảo tính liên kết chặt chẽ, kiến trúc dữ liệu của hệ thống được quy hoạch dựa trên hai trục trung tâm:

- **Trục Doanh nghiệp (Merchant):** Đóng vai trò là gốc của cấu trúc đa khách thuê (Multi-tenant). Mọi dữ liệu về chiến dịch (Campaign), người dùng (User), cấu hình API (APICredential) đều được định tuyến và cô lập dựa trên khóa ngoại MerchantID.
- **Trục Sổ cái Giao dịch (Ledger):** Tập trung vào thực thể CreditTransaction. Đây là rốn vũ trụ ghi nhận mọi biến động điểm thưởng, liên kết chặt chẽ với người dùng cuối (Customer), chiến dịch (Campaign) và chuỗi khối Sui để phục vụ đối soát.

2.1.1 Phân hệ Tổ chức và Phân quyền (Tenant & RBAC)

- **Thực thể Tenant (Đơn vị thuê bao gốc):**
 - TenantID (PK): Mã định danh duy nhất của công ty mẹ/chuỗi.
 - TenantName (NOT NULL): Tên định danh gốc.
 - SystemConfig (NULL): Cấu hình chung dạng JSON.
- **Thực thể Merchant (Doanh nghiệp đối tác):**
 - MerchantID (PK): Mã định danh cửa hàng/chi nhánh.
 - *TenantID* (FK, NOT NULL): Liên kết tới bảng Tenant.
 - BusinessName (NOT NULL): Tên thương hiệu/cửa hàng.
 - Status (NOT NULL): Trạng thái hoạt động (Active/Suspended).
- **Thực thể USER (Người dùng/Nhân viên):**
 - UserID (PK): Mã định danh nhân viên.

- *MerchantID* (FK, NOT NULL): Mã cửa hàng trực thuộc.
- *RoleID* (FK, NOT NULL): Mã phân quyền.
- *ManagerID* (FK, NULL): Mã người quản lý trực tiếp (tự tham chiếu).
- Email (UNIQUE): Email đăng nhập hệ thống.
- PasswordHash (NOT NULL): Mật khẩu đã được mã hóa.

- **Thực thể Role (Vai trò):**

- RoleID (PK): Mã định danh vai trò.
- RoleName (NOT NULL): Tên vai trò (Admin, Manager, Staff).

2.1.2 Phân hệ Tín dụng và Điểm thưởng (Loyalty Engine)

- **Thực thể Campaign (Chiến dịch điểm thưởng):**

- CampaignID (PK): Mã định danh chiến dịch.
- *TenantID* (FK, NOT NULL): Chuỗi (Tenant) sở hữu chiến dịch.
- CampaignName (NOT NULL): Tên chiến dịch.
- StartDate (NOT NULL): Ngày bắt đầu.
- EndDate (NULL): Thời điểm kết thúc.
- Status (NOT NULL): Trạng thái (Draft/Active/Expired).

- **Thực thể Campaign_Merchant (Phạm vi áp dụng):**

- CampaignID (PK, FK): Tham chiếu chiến dịch.
- MerchantID (PK, FK): Tham chiếu cửa hàng được áp dụng.

- **Thực thể RewardRule (Quy tắc tính điểm):**

- RuleID (PK): Mã định danh quy tắc.
- *CampaignID* (FK, NOT NULL): Thuộc chiến dịch nào.
- ActionType (NOT NULL): Loại hành động.
- PointsMultiplier (NOT NULL): Hệ số nhân điểm.

- **Thực thể CUSTOMER (Khách hàng):**

- CustomerID (PK): Mã định danh khách hàng nội bộ.
- WalletAddress (UNIQUE, NULL): Địa chỉ blockchain (Sui).
- Email (NULL): Địa chỉ email.

- **Thực thể Credit (Số dư ví điểm chung):**

- CreditID (PK): Mã định danh bản ghi số dư.
- *CustomerID* (FK, NOT NULL): Liên kết khách hàng.
- *TenantID* (FK, NOT NULL): Thuộc chuỗi (Tenant) nào.
- AvailableBalance (NOT NULL): Số dư khả dụng hiện tại.

2.1.3 Phân hệ Sổ cái Giao dịch và Kiểm toán (Ledger & Audit)

- **Thực thể CreditTransaction (Giao dịch điểm):**
 - TransactionID (PK): Mã định danh giao dịch nội bộ.
 - *CreditID* (FK, NOT NULL): Liên kết tới ví số dư.
 - *CampaignID* (FK, NULL): Phát sinh từ chiến dịch nào.
 - *MerchantID* (FK, NOT NULL): Cửa hàng nơi phát sinh giao dịch.
 - *Amount* (NOT NULL): Số điểm biến động.
 - *TransactionType* (NOT NULL): Loại (Earn/Redeem/Refund).
 - *SuiTxDigest* (UNIQUE, NULL): Mã băm đối soát trên chuỗi khối Sui.
 - *CreatedAt* (NOT NULL): Thời điểm giao dịch.
- **Thực thể APICredential (Khóa tích hợp):**
 - KeyID (PK): Mã định danh khóa.
 - *MerchantID* (FK, NOT NULL): Doanh nghiệp sở hữu khóa.
 - *ApiKey* (NOT NULL, UNIQUE): Mã định danh khóa công khai.
 - *SecretHash* (NOT NULL): Mã băm của Secret Key.

2.2 Xác định các mối quan hệ (Relationships)

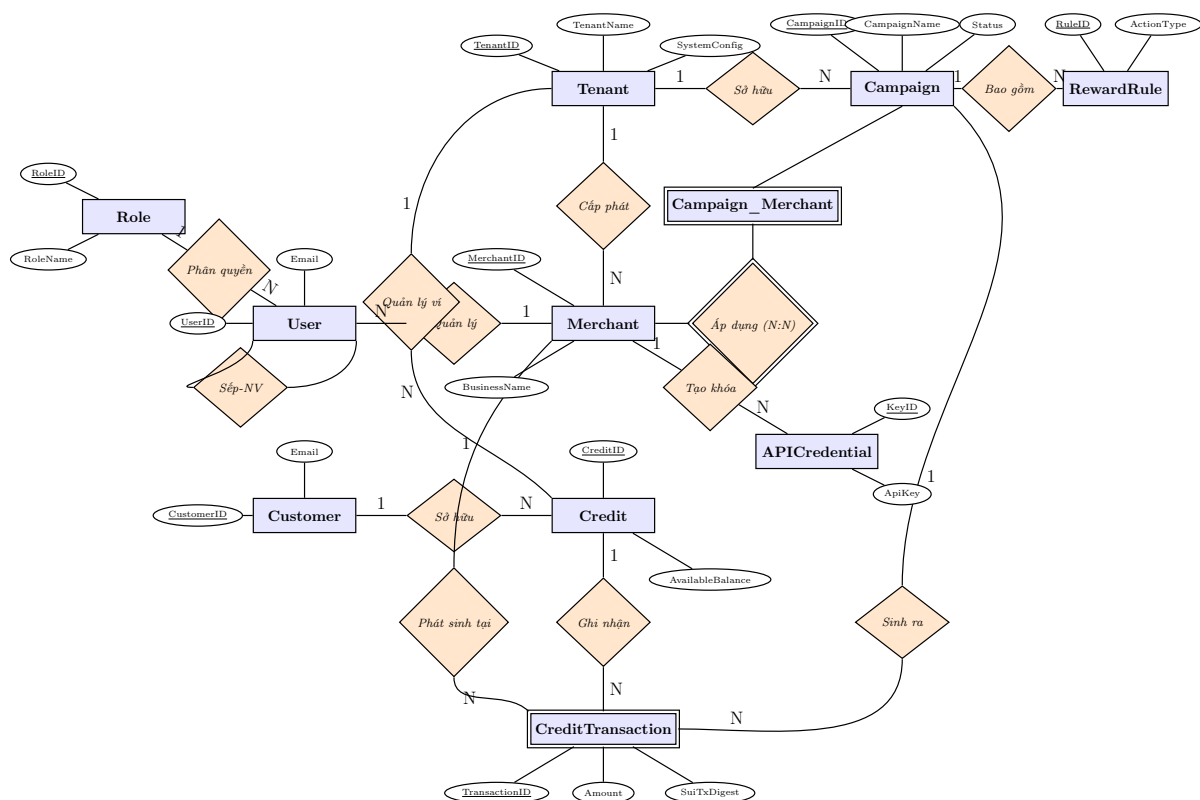
Dựa trên quy trình nghiệp vụ của hệ thống OctaPoint CaaS, các mối quan hệ (Relationships) giữa các thực thể được xác định và phân tích bản số (Cardinality) chi tiết như sau:

- **Tenant - Merchant (1:N):** Một công ty mẹ (Tenant) quản lý nhiều cửa hàng (Merchant).
- **Tenant - Campaign (1:N):** Công ty mẹ (Tenant) sở hữu và thiết lập các chiến dịch (Campaign) chung cho toàn hệ thống.
- **Campaign - Merchant (N:N):** Một chiến dịch có thể áp dụng cho nhiều cửa hàng, và một cửa hàng có thể chạy nhiều chiến dịch cùng lúc. Quan hệ này được mã hóa qua thực thể *Campaign_Merchant*.
- **Campaign - RewardRule (1:N):** Một chiến dịch bao gồm nhiều quy tắc điểm thưởng.
- **Merchant - User (1:N):** Một cửa hàng quản lý nhiều nhân sự (User) trực thuộc.
- **Customer - Credit (1:N) và Tenant - Credit (1:N):** Mỗi khách hàng có duy nhất 1 ví điểm (Credit) tại 1 chuỗi (Tenant).
- **Merchant - APICredential (1:N):** Cửa hàng sở hữu các mã khóa để tích hợp API.
- **Credit - CreditTransaction (1:N):** Một ví điểm ghi nhận nhiều biến động giao dịch.

- **Merchant - CreditTransaction (1:N):** Mọi giao dịch điểm thưởng đều phải lưu vết rõ cửa hàng (Merchant) nơi khách hàng thực hiện.
- **Campaign - CreditTransaction (1:N):** Giao dịch có thể sinh ra từ một chiến dịch.

2.3 Sơ đồ Thực thể Kết hợp (ERD)

Dựa trên danh sách các thực thể và mối quan hệ đã phân tích, sơ đồ ERD ở mức quan niệm được thiết kế theo ký pháp chuẩn Chen (Thực thể dạng hình chữ nhật, Mối kết hợp dạng hình thoi, Thuộc tính dạng hình bầu dục). Sơ đồ này độc lập với hệ quản trị CSDL vật lý.



Hình 2.1: Sơ đồ Thực thể Kết hợp (ERD) mức quan niệm hệ thống OctaPoint CaaS (Kiến trúc Ví điểm dùng chung)

Chương 3

Thiết kế Cơ sở dữ liệu Logic và Chuẩn hóa

Trong chương này, dự án sẽ tiến hành chuyển đổi từ mô hình thực thể quan niệm (ERD) sang lược đồ cơ sở dữ liệu quan hệ (Logic). Quá trình này được thực hiện thông qua các thuật toán ánh xạ, sau đó áp dụng lý thuyết phụ thuộc hàm để chứng minh các lược đồ đạt dạng chuẩn Boyce-Codd (BCNF), đảm bảo tính toàn vẹn dữ liệu và tối ưu hóa lưu trữ trước khi chuyển sang thiết kế vật lý.

3.1 Chuyển đổi từ Mô hình Quan niệm sang Lược đồ Logic

Dựa trên sơ đồ thực thể kết hợp (ERD) và các quy tắc nghiệp vụ đã phân tích ở Chương 2, tiến hành chuyển đổi mô hình thực thể quan niệm sang mô hình quan hệ (Relational Schema). Quá trình này được thực hiện thông qua việc áp dụng nghiêm ngặt các thuật toán ánh xạ ER sang quan hệ.

3.1.1 Các quy tắc chuyển đổi áp dụng

Quá trình ánh xạ từ ERD sang Lược đồ Logic tuân thủ các quy tắc cốt lõi sau:

- **Quy tắc 1 (Ánh xạ Thực thể mạnh):** Mỗi thực thể (Entity) mạnh trong ERD được chuyển thành một quan hệ (Bảng - Table) trong Lược đồ Logic. Các thuộc tính đơn trở thành các cột (Columns) của bảng. Khóa định danh của thực thể trở thành Khóa chính (Primary Key - PK).
- **Quy tắc 2 (Ánh xạ mối kết hợp 1:N):** Đối với các mối kết hợp một-nhiều (ví dụ: Merchant - Campaign, Customer - Credit), đưa khóa chính của bảng phía "1" sang bảng phía "N" làm khóa ngoại (Foreign Key - FK).
- **Quy tắc 3 (Ánh xạ mối kết hợp M:N):** Tạo một quan hệ trung gian cho mỗi kết hợp nhiều-nhiều (ví dụ quá trình ghi nhận giao dịch điểm). Đưa khóa của hai thực thể vào làm khóa ngoại. Khóa chính thường là tổ hợp của các khóa ngoại này cùng với thuộc tính định danh riêng (nếu có).

3.1.2 Danh sách Lược đồ Quan hệ sơ bộ

Dựa trên ERD của OctaPoint CaaS, áp dụng các quy tắc ánh xạ, ta thu được tập hợp 11 lược đồ quan hệ sơ bộ như sau. (Quy ước: Thuộc tính gạch chân là Khóa chính - PK, thuộc tính in nghiêng là Khóa ngoại - FK).

Tenant (TenantID, TenantName, SystemConfig)

Merchant (MerchantID, *TenantID*, BusinessName, Status)

Role (RoleID, RoleName)

User (UserID, *MerchantID*, Email, PasswordHash, *RoleID*, *ManagerID*)

Campaign (CampaignID, *TenantID*, CampaignName, StartDate, EndDate, Status)

Campaign_Merchant (*CampaignID*, *MerchantID*)

RewardRule (RuleID, *CampaignID*, ActionType, PointsMultiplier)

Customer (CustomerID, WalletAddress, Email)

Credit (CreditID, *CustomerID*, *TenantID*, AvailableBalance)

CreditTransaction (TransactionID, *CreditID*, *CampaignID*, *MerchantID*, Amount, TransactionType, SuiTxDigest, CreatedAt)

APICredential (KeyID, *MerchantID*, ApiKey, SecretHash)

3.2 Phụ thuộc hàm và Khóa ứng viên

Dựa trên lý thuyết thiết kế cơ sở dữ liệu quan hệ, để đảm bảo hệ thống OctaPoint không gặp phải các dị thường dữ liệu (Anomalies) khi Thêm, Xóa, Sửa, chúng ta cần phân tích tập Phụ thuộc hàm và tiến hành chuẩn hóa dữ liệu.

Xác định Tập Phụ thuộc hàm (Functional Dependencies)

Gọi U là tập hợp tất cả các thuộc tính của hệ thống. Dựa trên các quy tắc nghiệp vụ, ta xác định được tập Phụ thuộc hàm F cốt lõi như sau:

TenantID	→	TenantName, SystemConfig	(F1)
MerchantID	→	TenantID, BusinessName, Status	(F2)
UserID	→	MerchantID, Email, PasswordHash, RoleID, ManagerID	(F3)
Email	→	UserID, MerchantID, PasswordHash, RoleID	(F4)
CampaignID	→	TenantID, CampaignName, StartDate, EndDate, Status	(F5)
CreditID	→	CustomerID, TenantID, AvailableBalance	(F6)
TransactionID	→	CreditID, CampaignID, MerchantID, Amount, TransactionType, SuiTxDigest, CreatedAt	(F7)

Giải thích nghiệp vụ:

- Phụ thuộc hàm (F4) tồn tại vì email đăng nhập của User là duy nhất.
- (F5) và (F6) thể hiện rõ ví điểm và chiến dịch nay thuộc quyền sở hữu cấp cao nhất là Tenant.
- (F7) Giao dịch điểm bắt buộc định danh được Merchant nơi phát sinh.
- Bảng trung gian **Campaign_Merchant** chỉ chứa toàn bộ là khóa chính (CampaignID, MerchantID) nên không phát sinh phụ thuộc hàm không tầm thường.

3.2.1 Tính Bao đóng và Tìm khóa ứng viên

Tiến hành kiểm tra việc tìm khóa cho lược đồ **User**(UserID, MerchantID, Email, PasswordHash, RoleID, ManagerID) bằng thuật toán tính Bao đóng: Tập thuộc tính của lược đồ: $U_{User} = \{UserID, MerchantID, Email, PasswordHash, RoleID, ManagerID\}$.

Bước 1: Chọn tập thuộc tính ban đầu

Thử nghiệm với thuộc tính UserID. Khởi tạo $X = \{UserID\}$. Tính X^+ .

Bước 2: Quét qua tập Phụ thuộc hàm

Áp dụng (F3): $UserID \rightarrow MerchantID, Email, PasswordHash, RoleID, ManagerID$.

Ta có về trái $UserID \subseteq X^+$. Do đó: $X^+ = X \cup \{MerchantID, Email, PasswordHash, RoleID, ManagerID\} = U_{User}$.

Kết luận: Vì $\{UserID\}^+ = U_{User}$, nên **UserID** là một **Siêu khóa (Superkey)**. Hơn nữa, do tập $\{UserID\}$ chỉ có một phần tử duy nhất, không thể loại bỏ thêm phần tử nào mà vẫn giữ nguyên tính chất siêu khóa. Suy ra **UserID** là một **Khóa ứng viên**.

Thực hiện tương tự với $\{Email\}^+$ qua (F4), ta thu được $\{Email\}^+ = U_{User}$. Suy ra **Email** cũng là một **Khóa ứng viên**. Hệ thống chọn UserID làm Khóa chính (Primary Key).

3.3 Chuẩn hóa Dữ liệu (Normalization)

3.3.1 Kiểm chứng Dạng chuẩn Boyce-Codd (BCNF)

Theo định nghĩa, một lược đồ đạt chuẩn BCNF nếu mọi phụ thuộc hàm không tầm thường $X \rightarrow Y$ thì X phải là một Siêu khóa. Trong quan hệ **User**, hai phụ thuộc hàm tồn tại đều có về trái (UserID và Email) là Khóa ứng viên (và cũng là Siêu khóa). Do đó, lược đồ **User** đạt chuẩn BCNF.

Áp dụng phương pháp phân tích bao đóng tương tự cho các lược đồ còn lại, ta nhận thấy:

- Mọi phụ thuộc hàm $X \rightarrow Y$ đều có về trái X là Siêu khóa (dạng UUID hoặc mã tự tăng).
- Không tồn tại các phụ thuộc bắc cầu (Transitive Dependency) hay phụ thuộc từng phần (Partial Dependency), thỏa mãn điều kiện 3NF.

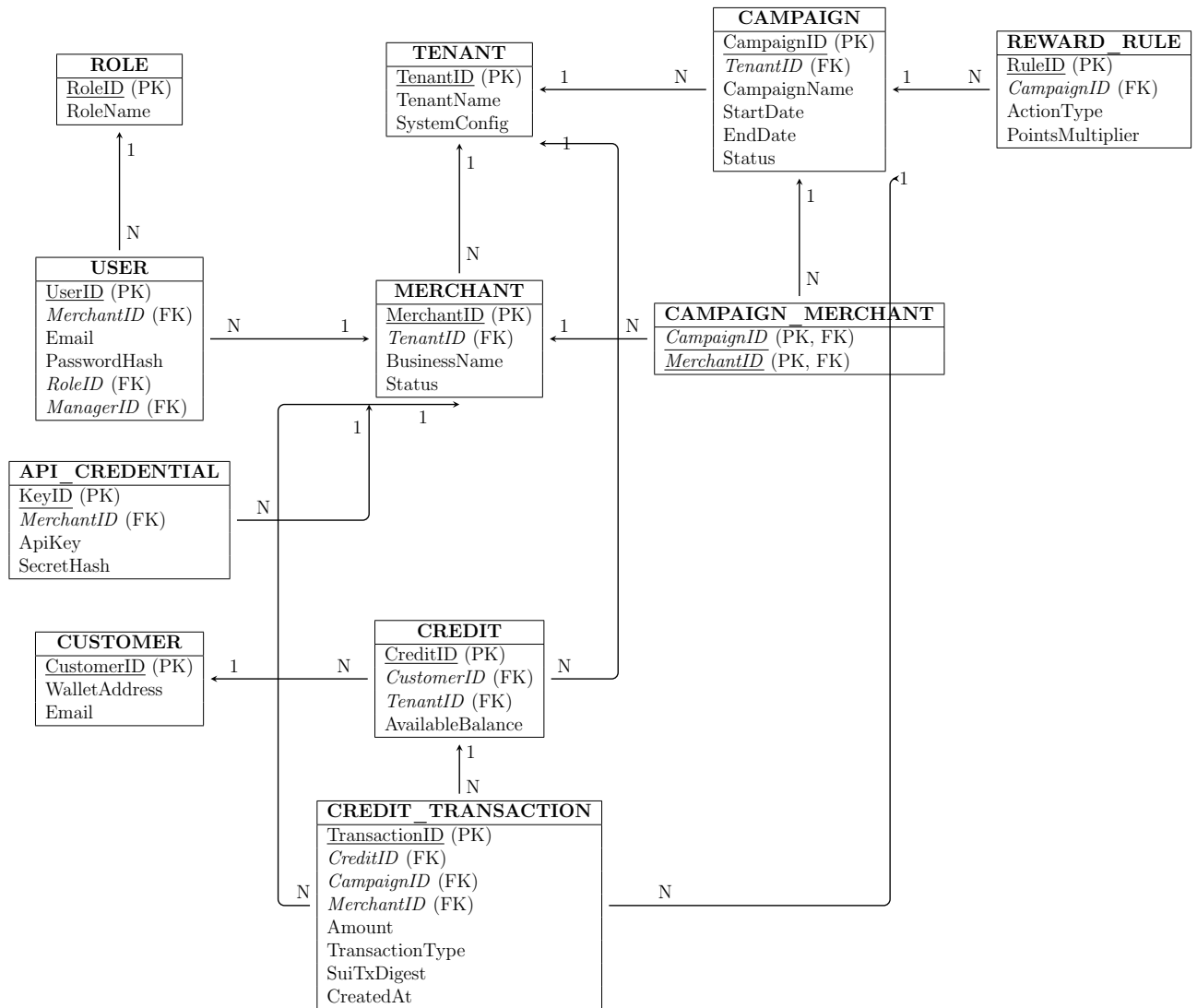
Mô hình logic sơ bộ đã thiết kế triệt tiêu được dị thường cập nhật và chính thức đạt Dạng chuẩn BCNF.

3.4 Sơ đồ Lược đồ Logic (Logical Schema Diagram)

Sau khi trải qua quá trình phân tích phụ thuộc hàm và chứng minh đạt Dạng chuẩn Boyce-Codd (BCNF), tập lược đồ quan hệ đã đảm bảo tính toàn vẹn, không dư thừa và tối ưu cho việc lưu trữ.

Đặc biệt, kiến trúc mới đã loại bỏ việc ví điểm bị trói buộc vào từng cửa hàng lẻ. Thay vào đó, ví điểm (CREDIT) được quản lý tập trung theo cấp chuỗi (TENANT). Đồng thời, mối kết hợp Nhiều - Nhiều (M:N) giữa CAMPAIGN và MERCHANT đã được phân rã thành hai mối quan hệ 1-N thông qua thực thể trung gian là bảng CAMPAIGN_MERCHANT.

Dưới đây là Sơ đồ Logic thể hiện trực quan cấu trúc các bảng, tập hợp đầy đủ các thuộc tính, khóa chính (Primary Key - PK) và các mối quan hệ khóa ngoại (Foreign Key - FK) kèm bản số (1-N) liên kết giữa các bảng.



Hình 3.1: Sơ đồ Lược đồ Logic chi tiết hệ thống OctaPoint CaaS

Ghi chú: Sơ đồ thể hiện rõ việc quản lý ví điểm tập trung tại TENANT và phân rã mối kết hợp N-N giữa Chiến dịch và Doanh nghiệp thông qua thực thể trung gian CAMPAIGN_MERCHANT. Các đường liên kết chỉ hướng tham chiếu từ Khóa ngoại (nhánh N) trở tới Khóa chính (nhánh 1), đảm bảo chặt chẽ ràng buộc toàn vẹn tham chiếu.

3.5 Từ điển Dữ liệu (Data Dictionary)

Từ điển dữ liệu (Data Dictionary) mô tả chi tiết cấu trúc vật lý của các bảng trong cơ sở dữ liệu. Nó bao gồm tên các thuộc tính, kiểu dữ liệu, các ràng buộc toàn vẹn (Khóa chính - PK, Khóa ngoại - FK, Not Null) và diễn giải ý nghĩa của từng trường dữ liệu để chuẩn bị cho bước triển khai vật lý ở Chương 4.

3.5.1 Bảng TENANT (Đơn vị thuê bao)

Lưu trữ thông tin về các đơn vị khách hàng lớn thuê hệ thống OctaPoint CaaS.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
TenantID	VARCHAR(36)	PK, NOT NULL	Mã định danh duy nhất (UUID).
TenantName	VARCHAR(255)	NOT NULL	Tên định danh của đơn vị thuê bao.
SystemConfig	JSON	Tùy chọn	Cấu hình hệ thống riêng của Tenant.

3.5.2 Bảng MERCHANT (Doanh nghiệp đối tác)

Lưu trữ thông tin các doanh nghiệp/cửa hàng trực thuộc một Tenant.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MerchantID	VARCHAR(36)	PK, NOT NULL	Mã định danh duy nhất của Merchant.
TenantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến bảng TENANT.
BusinessName	VARCHAR(255)	NOT NULL	Tên thương hiệu/doanh nghiệp.
Status	VARCHAR(20)	NOT NULL	Trạng thái hoạt động (Active, Inactive).

3.5.3 Bảng USER (Tài khoản nhân sự)

Quản lý tài khoản đăng nhập của nhân viên thuộc các Merchant.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
UserID	VARCHAR(36)	PK, NOT NULL	Mã định danh tài khoản.
MerchantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến bảng MERCHANT.
Email	VARCHAR(255)	UNIQUE	Email đăng nhập (duy nhất).
PasswordHash	VARCHAR(255)	NOT NULL	Chuỗi băm mật khẩu bảo mật.
RoleID	INT	FK, NOT NULL	Mã quyền hạn của người dùng.
ManagerID	VARCHAR(36)	FK, NULL	Mã người quản lý trực tiếp (tham chiếu đến UserID).

3.5.4 Bảng ROLE (Vai trò)

Lưu trữ thông tin các nhóm quyền hạn để phân quyền người dùng trong hệ thống.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
RoleID	INT	PK, NOT NULL	Mã định danh vai trò.
RoleName	NVARCHAR(50)	NOT NULL	Tên hiển thị của vai trò (VD: Admin, Staff).

3.5.5 Bảng CUSTOMER (Khách hàng)

Quản lý thông tin định danh của khách hàng tích điểm.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
CustomerID	VARCHAR(36)	PK, NOT NULL	Mã định danh khách hàng nội bộ.
WalletAddress	VARCHAR(100)	UNIQUE, NULL	Địa chỉ ví blockchain (Sui).
Email	VARCHAR(255)	NULL	Địa chỉ email (Định danh Web2).

3.5.6 Bảng CREDIT (Số dư ví điểm)

Lưu trữ số dư khả dụng của khách hàng. Mỗi khách hàng chỉ có một ví điểm duy nhất tại một chuỗi.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
CreditID	VARCHAR(36)	PK, NOT NULL	Mã định danh bản ghi số dư.
CustomerID	VARCHAR(36)	FK, NOT NULL	Tham chiếu khách hàng.
TenantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu chuỗi (Tenant).
AvailableBalance	DECIMAL(18,2)	NOT NULL	Số dư khả dụng hiện tại.

3.5.7 Bảng CAMPAIGN (Chiến dịch)

Quản lý các chương trình khuyến mãi, tích/đổi điểm cấp toàn chuỗi.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
CampaignID	VARCHAR(36)	PK, NOT NULL	Mã định danh chiến dịch.
TenantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến bảng TENANT.
CampaignName	NVARCHAR(255)	NOT NULL	Tên chiến dịch.
StartDate	DATETIME	NOT NULL	Thời gian bắt đầu.
EndDate	DATETIME	NULL	Thời điểm kết thúc.
Status	VARCHAR(20)	NOT NULL	Trạng thái (Draft, Active, Ended).

3.5.8 Bảng CAMPAIGN_MERCHANT (Phạm vi áp dụng)

Bảng trung gian quy định chiến dịch nào được phép chạy ở cửa hàng nào.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
CampaignID	VARCHAR(36)	PK, FK	Tham chiếu chiến dịch.
MerchantID	VARCHAR(36)	PK, FK	Tham chiếu cửa hàng.

3.5.9 Bảng REWARD_RULE (Quy tắc tính điểm)

Chi tiết cấu hình quy tắc và hệ số nhân điểm của từng chiến dịch.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
RuleID	VARCHAR(36)	PK, NOT NULL	Mã định danh quy tắc.
CampaignID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến chiến dịch.
ActionType	VARCHAR(50)	NOT NULL	Loại hành động (VD: Mua hàng).
PointsMultiplier	DECIMAL(5,2)	NOT NULL	Hệ số nhân điểm thưởng.

3.5.10 Bảng CREDIT_TRANSACTION (Sổ cái giao dịch điểm)

Lưu vết toàn bộ các giao dịch cộng/trừ điểm thưởng, có lưu băm trên blockchain.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
TransactionID	VARCHAR(36)	PK, NOT NULL	Mã giao dịch định danh.
CreditID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến ví điểm.
CampaignID	VARCHAR(36)	FK, NULL	Tham chiếu đến chiến dịch.
MerchantID	VARCHAR(36)	FK, NOT NULL	Cửa hàng phát sinh giao dịch.
Amount	DECIMAL(18,2)	NOT NULL	Số lượng điểm biến động.
TransactionType	VARCHAR(50)	NOT NULL	Loại giao dịch (Earn, Burn).
SuiTxDigest	VARCHAR(100)	UNIQUE, NULL	Mã băm trên mạng lưới Sui.
CreatedAt	DATETIME	NOT NULL	Thời gian phát sinh giao dịch.

3.5.11 Bảng API_CREDENTIAL (Khóa tích hợp)

Cung cấp khóa API để tích hợp phần mềm bán hàng của cửa hàng với nền tảng.

Tên thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
KeyID	VARCHAR(36)	PK, NOT NULL	Mã định danh khóa.
MerchantID	VARCHAR(36)	FK, NOT NULL	Tham chiếu đến bảng MERCHANT.
ApiKey	VARCHAR(255)	UNIQUE, NOT NULL	Mã định danh khóa công khai.
SecretHash	VARCHAR(255)	NOT NULL	Chuỗi băm của Secret Key.

Chương 4

Thiết kế Vật lý và Triển khai

Theo quy trình thiết kế cơ sở dữ liệu, thiết kế vật lý không làm thay đổi ý nghĩa dữ liệu mà quyết định cách Hệ quản trị CSDL (DBMS) lưu trữ, đọc, ghi và tìm kiếm dữ liệu tối ưu nhất dựa trên tải công việc (workload).

Chương này sử dụng lược đồ logic đã chuẩn hóa và các đặc tả workload làm đầu vào, từ đó tiến hành lựa chọn tổ chức lưu trữ, đánh chỉ mục và xuất ra cấu trúc mã lệnh DDL vật lý.

4.1 Đặc tả Workload và Chọn tổ chức lưu trữ

Thiết kế vật lý bắt buộc phải dựa vào tải công việc (workload) thực tế của hệ thống để đưa ra quyết định tổ chức tập tin cơ bản (Heap, Sorted hay Hash).

4.1.1 Đặc tả tải công việc (Workload)

Hệ thống OctaPoint CaaS có đặc thù tải công việc như sau:

- **Ghi liên tục (Write-heavy):** Bảng CREDIT_TRANSACTION liên tục nhận dữ liệu mới khi khách hàng giao dịch điểm.
- **Tra cứu chính xác và theo khoảng (Range Query):** Việc tra cứu số dư ví điểm (CREDIT) hoặc liệt kê các chiến dịch (CAMPAIGN) diễn ra trong một khoảng thời gian nhất định chiếm tần suất cao.

4.1.2 Lựa chọn tổ chức lưu trữ (Storage Organization)

Dựa trên yêu cầu workload, nếu sử dụng tổ chức Heap (dữ liệu chèn ở cuối) sẽ làm chậm quá trình tìm kiếm, trong khi tổ chức Hash chỉ phù hợp tìm kiếm chính xác ($O(1)$) nhưng không hỗ trợ tìm kiếm theo khoảng.

Do đó, dự án quyết định lựa chọn hệ quản trị **SQL Server** vì các đặc tính vật lý sau:

- **Tổ chức Clustered Index (B-Tree):** SQL Server lưu trữ dữ liệu các bảng theo cấu trúc cây B-Tree dựa trên Khóa chính (Clustered Index). Điều này hỗ trợ truy vấn theo khoảng cực kỳ hiệu quả nhờ các nút lá được liên kết với nhau.
- **Tính toàn vẹn (ACID):** Đảm bảo an toàn tuyệt đối cho các giao dịch điểm thưởng nhiều bước.

- **Lưu trữ theo khối (Page/Extent):** Kiến trúc lưu trữ theo từng Page (8KB) giúp tối ưu hóa hệ số chặn (Blocking factor - *bfr*) khi nạp dữ liệu từ đĩa cứng vào bộ đệm (Buffer pool), giúp giảm thiểu tối đa số lần I/O.

4.2 Ánh xạ Kiểu dữ liệu và Ràng buộc (Đầu vào)

Theo quy trình xử lý, đầu vào của thiết kế vật lý bao gồm Lược đồ logic đã được chuẩn hóa (đạt chuẩn BCNF ở Chương 3), cùng với tập hợp các khóa và ràng buộc. Hệ thống OctaPoint tiến hành ánh xạ các yếu tố này sang hệ quản trị SQL Server như sau:

4.2.1 Ánh xạ Kiểu dữ liệu (Data Type Mapping)

- **Khóa định danh (ID):** Sử dụng VARCHAR(36) để lưu UUID thay vì số nguyên tự tăng (INT IDENTITY) nhằm hỗ trợ sinh khóa phân tán và bảo mật.
- **Dữ liệu tài chính:** Các cột Amount, AvailableBalance bắt buộc dùng kiểu DECIMAL(18,2) để tránh sai số dấu phẩy động trong quá trình cộng/trừ điểm thưởng.
- **Dữ liệu phi cấu trúc:** Cột SystemConfig (bảng TENANT) sử dụng kiểu NVARCHAR(MAX) để lưu trữ cấu trúc JSON, tận dụng khả năng phân tích chuỗi động tích hợp sẵn của SQL Server.

4.2.2 Ràng buộc Toàn vẹn (Constraints)

- **Khóa chính (Primary Key - PK):** Bắt buộc tồn tại trên mọi bảng để xác định duy nhất một bản ghi trong khối dữ liệu (SQL Server sẽ tự động tạo Clustered Index trên cột này).
- **Khóa ngoại (Foreign Key - FK):** Thiết lập ràng buộc tham chiếu chặt chẽ (mặc định ON DELETE NO ACTION) để ngăn chặn việc tạo ra dữ liệu mồ côi (orphan data) khi xóa các thực thể cha như Merchant hay Campaign.
- **Ràng buộc Unique (Tính duy nhất):** Áp dụng cho Email (bảng USER), SuiTxDigest (bảng CREDIT_TRANSACTION) để chống trùng lặp. Đặc biệt, thiết lập ràng buộc UNIQUE(CustomerID, TenantID) trên bảng CREDIT để đảm bảo tuân thủ tuyệt đối quy tắc nghiệp vụ: Mỗi khách hàng chỉ có duy nhất 1 ví điểm tại 1 chuỗi bán lẻ.

4.3 Chiến lược Đánh chỉ mục (Indexing)

Chỉ mục (Index) là cấu trúc phụ giúp DBMS tìm khối dữ liệu cần đọc nhanh chóng mà không phải quét toàn bộ bảng. Tuy nhiên, việc lạm dụng chỉ mục sẽ gây tốn không gian lưu trữ và làm chậm các thao tác chèn/cập nhật dữ liệu.

4.3.1 Cơ sở lý thuyết và Thuật toán lựa chọn

Dựa trên **Algorithm 4.1 (Quyết định tạo chỉ mục)**, dự án áp dụng các nguyên tắc sau:

1. **Ưu tiên Khóa:** Luôn xem xét tạo chỉ mục trên Khóa chính (PK) và Khóa ngoại (FK) trước.
2. **Đánh giá Độ chọn lọc (Selectivity):** Chỉ mục cây B-Tree chỉ hiệu quả khi cột có độ chọn lọc cao (càng nhiều giá trị phân biệt càng tốt). Công thức tính độ chọn lọc được định nghĩa là:

$$selectivity(A) = \frac{NDV(A)}{|R|}$$

Trong đó, $NDV(A)$ là số giá trị phân biệt của cột A , và $|R|$ là tổng số dòng của bảng.

3. **Chi phí ghi:** Hạn chế tối đa việc tạo chỉ mục phụ trên bảng `CREDIT_TRANSACTION` vì đây là bảng có tải công việc ghi liên tục (Write-heavy workload).

4.3.2 Thiết kế Chỉ mục cho OctaPoint CaaS

Từ các nguyên tắc trên, hệ thống triển khai các loại chỉ mục cụ thể như sau:

- **Chỉ mục Cụm (Clustered Index):** Được SQL Server tự động tạo trên Khóa chính của tất cả các bảng. Dữ liệu vật lý được tổ chức theo cấu trúc Cây B-Tree, hỗ trợ tìm kiếm chính xác và tìm kiếm theo khoảng (Range Query) với chi phí $O(\log_p N)$.
- **Chỉ mục Phụ (Nonclustered Index) trên Khóa ngoại:** Tạo chỉ mục cho các cột FK quan trọng như `TenantID`, `MerchantID`, `CreditID`, `CampaignID` nhằm tối ưu hóa chi phí cho thuật toán kết nối lồng nhau (Nested-Loop Join) khi thực hiện các truy vấn thống kê nhiều bảng.
- **Chỉ mục Phụ trên Ràng buộc UNIQUE:** SQL Server tự động tạo Nonclustered Index ngầm định trên các cột được gán ràng buộc UNIQUE. Nhờ vậy, hệ thống được tăng tốc cực nhanh khi truy vấn cột `Email` (bảng `USER`) lúc đăng nhập, hoặc cập (`CustomerID`, `TenantID`) (bảng `CREDIT`) lúc kiểm tra số dư ví chung.
- **Cột KHÔNG tạo chỉ mục:** Các cột như `Status` (chỉ có Active/Inactive) hay `TransactionType` (chỉ có Earn/Burn/Refund) có độ chọn lọc cực thấp. Việc dùng B-Tree cho các cột này không đáng giá vì mỗi giá trị trả về quá nhiều dòng. Dự án quyết định không đánh chỉ mục trên các cột này để tiết kiệm dung lượng lưu trữ đĩa và tối ưu tốc độ ghi dữ liệu.

4.4 Mã lệnh tạo bảng (DDL - Data Definition Language)

Dựa trên từ điển dữ liệu (Chương 3) và chiến lược thiết kế vật lý, dưới đây là mã lệnh SQL (DDL) sử dụng để khởi tạo cấu trúc cơ sở dữ liệu. Hệ quản trị được sử dụng là SQL Server.

```

1  -- 1. BANG DOC LAP (Khong chua khoa ngoai)
2  CREATE TABLE TENANT (
3      TenantID VARCHAR(36) PRIMARY KEY,
4      TenantName NVARCHAR(255) NOT NULL,
5      SystemConfig NVARCHAR(MAX) NULL
6  );
7  GO
8
9  CREATE TABLE ROLE (
10     RoleID INT PRIMARY KEY,
11     RoleName NVARCHAR(50) NOT NULL
12 );
13 GO
14
15 CREATE TABLE CUSTOMER (
16     CustomerID VARCHAR(36) PRIMARY KEY,
17     WalletAddress VARCHAR(100) UNIQUE NULL,
18     Email VARCHAR(255) NULL
19 );
20 GO
21
22 -- 2. BANG PHU THUOC CAP 1
23 CREATE TABLE MERCHANT (
24     MerchantID VARCHAR(36) PRIMARY KEY,
25     TenantID VARCHAR(36) NOT NULL,
26     BusinessName NVARCHAR(255) NOT NULL,
27     Status VARCHAR(20) NOT NULL,
28     CONSTRAINT FK_Merchant_Tenant FOREIGN KEY (TenantID) REFERENCES
29         TENANT(TenantID)
30 );
31 GO
32
33 CREATE TABLE CAMPAIGN (
34     CampaignID VARCHAR(36) PRIMARY KEY,
35     TenantID VARCHAR(36) NOT NULL,
36     CampaignName NVARCHAR(255) NOT NULL,
37     StartDate DATETIME NOT NULL,
38     EndDate DATETIME NULL,
39     Status VARCHAR(20) NOT NULL,
40     CONSTRAINT FK_Campaign_Tenant FOREIGN KEY (TenantID) REFERENCES
41         TENANT(TenantID)
42 );
43 GO
44
45 CREATE TABLE CREDIT (
46     CreditID VARCHAR(36) PRIMARY KEY,
47     CustomerID VARCHAR(36) NOT NULL,
48     TenantID VARCHAR(36) NOT NULL,
49     AvailableBalance DECIMAL(18,2) NOT NULL DEFAULT 0.00,
50     CONSTRAINT FK_Credit_Customer FOREIGN KEY (CustomerID) REFERENCES
51         CUSTOMER(CustomerID),
52     CONSTRAINT FK_Credit_Tenant FOREIGN KEY (TenantID) REFERENCES
53         TENANT(TenantID),
54     CONSTRAINT UQ_Customer_Tenant UNIQUE (CustomerID, TenantID) -- 1
55         Khach hang chi co 1 vi tai 1 chuoi
56 );
57 GO

```

```

53
54 -- 3. BANG PHU THUOC CAP 2
55 CREATE TABLE [USER] (
56     UserID VARCHAR(36) PRIMARY KEY,
57     MerchantID VARCHAR(36) NOT NULL,
58     Email VARCHAR(255) UNIQUE NOT NULL,
59     PasswordHash VARCHAR(255) NOT NULL,
60     RoleID INT NOT NULL,
61     ManagerID VARCHAR(36) NULL,
62     CONSTRAINT FK_User_Merchant FOREIGN KEY (MerchantID) REFERENCES
        MERCHANT(MerchantID),
63     CONSTRAINT FK_User_Role FOREIGN KEY (RoleID) REFERENCES ROLE(RoleID
        ),
64     CONSTRAINT FK_User_Manager FOREIGN KEY (ManagerID) REFERENCES [USER
        ](UserID)
65 );
66 GO
67
68 CREATE TABLE API_CREDENTIAL (
69     KeyID VARCHAR(36) PRIMARY KEY,
70     MerchantID VARCHAR(36) NOT NULL,
71     ApiKey VARCHAR(255) UNIQUE NOT NULL,
72     SecretHash VARCHAR(255) NOT NULL,
73     CONSTRAINT FK_API_Merchant FOREIGN KEY (MerchantID) REFERENCES
        MERCHANT(MerchantID)
74 );
75 GO
76
77 CREATE TABLE CAMPAIGN_MERCHANT (
78     CampaignID VARCHAR(36) NOT NULL,
79     MerchantID VARCHAR(36) NOT NULL,
80     PRIMARY KEY (CampaignID, MerchantID),
81     CONSTRAINT FK_CM_Campaign FOREIGN KEY (CampaignID) REFERENCES
        CAMPAIGN(CampaignID),
82     CONSTRAINT FK_CM_Merchant FOREIGN KEY (MerchantID) REFERENCES
        MERCHANT(MerchantID)
83 );
84 GO
85
86 -- 4. BANG PHU THUOC CAP 3
87 CREATE TABLE REWARD_RULE (
88     RuleID VARCHAR(36) PRIMARY KEY,
89     CampaignID VARCHAR(36) NOT NULL,
90     ActionType VARCHAR(50) NOT NULL,
91     PointsMultiplier DECIMAL(5,2) NOT NULL,
92     CONSTRAINT FK_Rule_Campaign FOREIGN KEY (CampaignID) REFERENCES
        CAMPAIGN(CampaignID)
93 );
94 GO
95
96 CREATE TABLE CREDIT_TRANSACTION (
97     TransactionID VARCHAR(36) PRIMARY KEY,
98     CreditID VARCHAR(36) NOT NULL,
99     CampaignID VARCHAR(36) NULL,
100     MerchantID VARCHAR(36) NOT NULL,
101     Amount DECIMAL(18,2) NOT NULL,
102     TransactionType VARCHAR(50) NOT NULL,
103     SuiTxDigest VARCHAR(100) UNIQUE NULL,

```

```

104 CreatedAt DATETIME NOT NULL DEFAULT GETDATE(),
105 CONSTRAINT FK_Trans_Credit FOREIGN KEY (CreditID) REFERENCES CREDIT
    (CreditID),
106 CONSTRAINT FK_Trans_Campaign FOREIGN KEY (CampaignID) REFERENCES
    CAMPAIGN(CampaignID),
107 CONSTRAINT FK_Trans_Merchant FOREIGN KEY (MerchantID) REFERENCES
    MERCHANT(MerchantID)
108 );
109 GO

```

4.5 Tối ưu hóa truy xuất dữ liệu

Dựa trên pipeline xử lý truy vấn của hệ quản trị CSDL, việc viết câu lệnh SQL là chưa đủ. Hệ thống cần tối ưu hóa cây truy vấn (Query Tree) thông qua các luật biến đổi tương đương để giảm thiểu chi phí bộ nhớ và I/O trước khi lập kế hoạch thực thi (Execution Plan).

4.5.1 Bài toán truy vấn

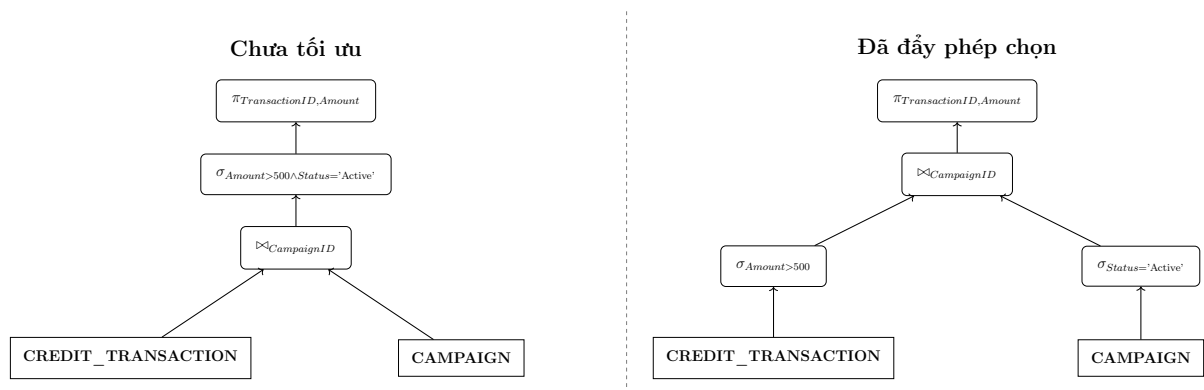
Yêu cầu: Lấy ra Mã giao dịch (TransactionID) và Số điểm (Amount) của các giao dịch cộng điểm lớn hơn 500, thuộc về các Chiến dịch (CAMPAIGN) đang ở trạng thái hoạt động ('Active').

Câu lệnh SQL đại số quan hệ:

$$\pi_{TransactionID, Amount}(\sigma_{Amount > 500 \wedge Status = 'Active'}(CREDIT_TRANSACTION \bowtie CAMPAIGN))$$

4.5.2 Cây truy vấn trước và sau khi tối ưu

Theo luật biến đổi tương đương, phép chọn (Selection - σ) nếu được đẩy xuống trước phép kết nối (Join - $\bowtie$) sẽ giúp giảm thiểu đáng kể kích thước dữ liệu trung gian.



Hình 4.1: Cây truy vấn trước và sau khi tối ưu hóa bằng luật phân rã

4.5.3 Đánh giá thuật toán kết nối (Join Algorithm)

Bằng việc đẩy phép chọn xuống dưới, kích thước của 2 bảng trước khi thực hiện phép Join đã bị thu hẹp đáng kể.

Tuy nhiên, thay vì khẳng định hệ thống luôn chọn một thuật toán cố định, ta cần hiểu rằng Trình tối ưu hóa truy vấn (Query Optimizer) của SQL Server sẽ đánh giá dựa trên thông kê thực tế. Nếu bảng **CAMPAIGN** sau khi lọc trở nên rất nhỏ và có thể nạp toàn bộ vào bộ nhớ đệm (Buffer pool), thuật toán **Nested-Loop Join** có thể được hệ thống ưu tiên. Ngược lại, nếu kích thước dữ liệu trung gian của cả hai bảng vẫn lớn và không có thứ tự, hệ thống sẽ linh hoạt chuyển sang sử dụng **Hash Join** để đạt chi phí $O(b_R + b_S)$.

4.6 Một số câu truy vấn mẫu

Dưới đây là các câu truy vấn T-SQL tiêu biểu để kiểm chứng thiết kế cơ sở dữ liệu trên hệ quản trị SQL Server, thể hiện rõ khả năng xử lý dữ liệu tập trung theo kiến trúc Multi-tenant.

4.6.1 Truy vấn 1: Xem lịch sử giao dịch điểm của một khách hàng tại một cửa hàng

```
1 SELECT
2     ct.TransactionID ,
3     ct.TransactionType ,
4     ct.Amount ,
5     ct.CreatedAt
6 FROM CREDIT_TRANSACTION ct
7 JOIN CREDIT c ON ct.CreditID = c.CreditID
8 WHERE c.CustomerID = 'CUST_UUID_001'
9     AND ct.MerchantID = 'MERCH_UUID_002'
10 ORDER BY ct.CreatedAt DESC;
```

4.6.2 Truy vấn 2: Tra cứu số dư điểm hiện tại của khách hàng trên toàn bộ các chuỗi (Tenant)

```
1 SELECT
2     t.TenantName ,
3     c.AvailableBalance
4 FROM CREDIT c
5 JOIN TENANT t ON c.TenantID = t.TenantID
6 WHERE c.CustomerID = 'CUST_UUID_001'
7 ORDER BY c.AvailableBalance DESC;
```

4.6.3 Truy vấn 3: Thống kê tổng số điểm đã phát hành của một chiến dịch tại một cửa hàng trong tháng

```
1 SELECT
2     camp.CampaignName ,
3     COUNT(ct.TransactionID) AS TotalTransactions ,
4     SUM(ct.Amount) AS TotalPointsIssued
5 FROM CREDIT_TRANSACTION ct
6 JOIN CAMPAIGN camp ON ct.CampaignID = camp.CampaignID
7 WHERE ct.MerchantID = 'MERCH_UUID_002'
```

```

8   AND ct.TransactionType = 'Earn'
9   AND MONTH(ct.CreatedAt) = MONTH(GETDATE())
10  AND YEAR(ct.CreatedAt) = YEAR(GETDATE())
11 GROUP BY camp.CampaignID, camp.CampaignName;

```

4.6.4 Truy vấn 4: Phân tích khách hàng (Top 5 người có số dư cao nhất tại một Chuỗi)

```

1  SELECT TOP 5
2      cust.Email,
3      cust.WalletAddress,
4      c.AvailableBalance
5  FROM CREDIT c
6  JOIN CUSTOMER cust ON c.CustomerID = cust.CustomerID
7  WHERE c.TenantID = 'TENANT_UUID_001'
8  ORDER BY c.AvailableBalance DESC;

```

4.6.5 Truy vấn 5: Thống kê hiệu suất các cửa hàng (Tổng điểm đã tiêu/tích lũy theo từng chi nhánh của chuỗi)

```

1  SELECT
2      m.BusinessName,
3      ct.TransactionType,
4      COUNT(ct.TransactionID) AS TotalTx,
5      SUM(ct.Amount) AS TotalPoints
6  FROM CREDIT_TRANSACTION ct
7  JOIN MERCHANT m ON ct.MerchantID = m.MerchantID
8  WHERE m.TenantID = 'TENANT_UUID_001'
9  GROUP BY m.BusinessName, ct.TransactionType
10 ORDER BY m.BusinessName, ct.TransactionType;

```

4.6.6 Truy vấn 6: Tra cứu danh sách các cửa hàng được phép áp dụng một Chiến dịch đang hoạt động

```

1  SELECT
2      camp.CampaignName,
3      m.BusinessName,
4      m.Status AS MerchantStatus
5  FROM CAMPAIGN_MERCHANT cm
6  JOIN CAMPAIGN camp ON cm.CampaignID = camp.CampaignID
7  JOIN MERCHANT m ON cm.MerchantID = m.MerchantID
8  WHERE camp.Status = 'Active'
9      AND camp.TenantID = 'TENANT_UUID_001'
10 ORDER BY camp.CampaignName, m.BusinessName;

```